

Movie Search na Spin/WASM i OCI

Porównanie WebAssembly jako izolacji dla mikroserwisów

Projekt LSC

24 maja 2026

Streszczenie

Projekt porównuje tę samą aplikację HTTP uruchomioną w dwóch modelach izolacji: Spin/wasmtime jako komponent WASI oraz natywny serwer Rust zapakowany jako obraz OCI. Po wstępnej analizie porzucono próbę porównywania Spin z pełnym Meilisearch, ponieważ oficjalny Meilisearch używa LMDB i założeń typowych dla native runtime, których nie dało się przenieść 1:1 do środowiska WASI w czasie projektu. Zamiast tego zbudowano własny, deterministyczny serwis `movie-search`, w którym Spin i OCI współdzielą ten sam kod aplikacyjny, fixture oraz semantykę API. Wyniki pokazują pełną zgodność funkcjonalną, krótszy pierwszy request wyszukiwania po stronie Spin, bardzo szybkie odpowiedzi OCI dla prostego pustego zapytania oraz wyraźnie większy obserwowany RSS procesu Spin pod obciążeniem. Wnioskiem jest, że Spin/WASM jest sensownym kandydatem dla izolowanych, krótkich mikroserwisów HTTP, ale kontenery OCI pozostają bardziej dojrzałym i przewidywalnym wyborem operacyjnym, zwłaszcza przy usługach storage-heavy.

1 Cel i kontekst

Tematem projektu było sprawdzenie, czy WebAssembly uruchamiane przez Spin i wasmtime może być praktycznym podłożem izolacji dla mikroserwisów typu serverless w porównaniu z klasycznym kontenerem OCI. WebAssembly jest binarnym formatem instrukcji projektowanym jako przenośny cel kompilacji dla języków programowania, także poza przeglądarką [1]. WASI rozszerza ten model o interfejsy systemowe, np. pliki, sieć, zegary i losowość, zachowując kontrolowany model uprawnień [3, 2]. Spin jest frameworkiem do budowania mikroserwisów i aplikacji WebAssembly, z prostym cyklem `spin build` oraz `spin up`, obsługą HTTP triggerów i języków zgodnych z WASI [4]. Wasmtime pełni w tym układzie rolę runtime dla WebAssembly, WASI i Component Model [3].

Punktem odniesienia jest OCI: otwarty standard opisu obrazów i uruchamiania kontenerów, zbudowany wokół specyfikacji runtime, image i distribution [5]. Kontener Docker pakuje aplikację oraz zależności w standaryzowaną jednostkę uruchomieniową, izolowaną od środowiska hosta i przenośną pomiędzy środowiskami [6]. Pytanie badawcze brzmi więc: co dostajemy, gdy ten sam kod aplikacyjny uruchomimy raz jako komponent WASI, a raz jako natywny proces w kontenerze?

2 Pivot z Meilisearch

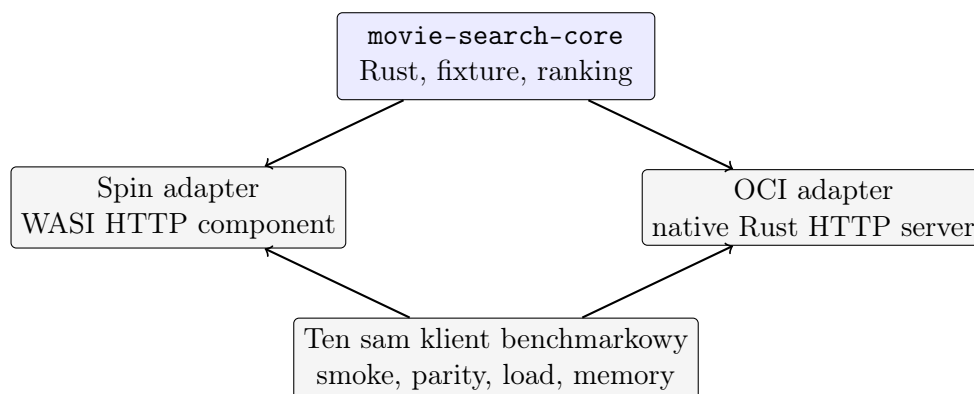
Pierwszy kierunek zakładał porównanie Meilisearch w kontenerze z wariantem uruchamianym przez Spin. Analiza feasibility pokazała jednak, że byłoby to porównanie nieuczciwe technicznie. Oficjalny Meilisearch przechowuje dane w `data.ms` i używa LMDB jako silnika storage; LMDB opiera się na pliku mapowanym w pamięci [7]. Backupy Meilisearch rozróżniają snapshoty, czyli kopię bazy `data.ms`, oraz dumpy przenośne między wersjami Meilisearch [8]. To nadal jest model danych Meilisearch, a nie przenośny indeks dla własnego komponentu WASI.

Dodatkowo lokalne próby kompilacji upstreamowych warstw Meilisearch do `wasm32-wasip2` zatrzymały się na zależnościach takich jak `ring`, Tokio oraz przewidywanych blokadach storage

związanych z LMDB i mmap. Dlatego projekt zmieniono z “portujemy Meilisearch” na “porównujemy ten sam własny mikroservis w dwóch izolacjach runtime”. Dzięki temu benchmark mierzy różnicę uruchomienia i izolacji, a nie różnicę semantyki wyszukiwarki.

3 Aplikacja i architektura

Końcowa aplikacja to `movie-search`: prosty serwis HTTP napisany w Rust. Oba runtime’y używają tej samej biblioteki `movie-search-core`, tego samego pliku `fixtures/movies.json` i tych samych reguł rankingu. Fixture zawiera 44471 unikalnych filmów po deduplikacji po polu `id`. Wyszukiwanie jest deterministyczne: tokenizacja po znakach niealfanumerycznych, dopasowanie przez case-insensitive substring, sortowanie po liczbie trafionych tokenów, wadze pól `title` > `genre` > `overview`, a na końcu po rosnącym `id`. Celowo nie zaimplementowano typy tolerance, synonimów ani reguł Meilisearch.



Rysunek 1: Architektura końcowa: jeden core aplikacyjny, dwa adaptory runtime.

Publiczne API jest wspólne dla obu wariantów: `GET /health`, `GET /version`, `GET /stats`, `GET /movies?offset=&limit=` oraz `POST /search`. Spin działa lokalnie na porcie 8080, a wariant OCI na porcie 8081.

4 Metodyka

Przed pomiarem wydajności uruchamiane są testy jednostkowe, build komponentu Spin, build obrazu OCI oraz smoke/parity checks. Skrypt `benchmarks/compare_results.sh` porównuje silnik, liczbę dokumentów, liczbę trafień i listy `hit.id` dla zapytań `space`, `toy story`, `dark knight`, `romance` i zapytania pustego.

Pełny benchmark uruchomiono komendą `benchmarks/run_all.sh`. Obejmuje on 20 powtórzeń cold start, testy obciążeniowe `POST /search` dla zapytań `space` i pustego przy współbieżności 10, 50, 100 i 200, oraz próbki pamięci w stanie idle i under-load. Środowisko: Apple M4, 10 rdzeni, 24 GiB RAM, macOS 15.6, Spin 3.6.3, Docker 28.5.1, Rust 1.95.0. Obraz Docker był budowany przed pomiarem cold start; czas budowania nie jest częścią wyniku.

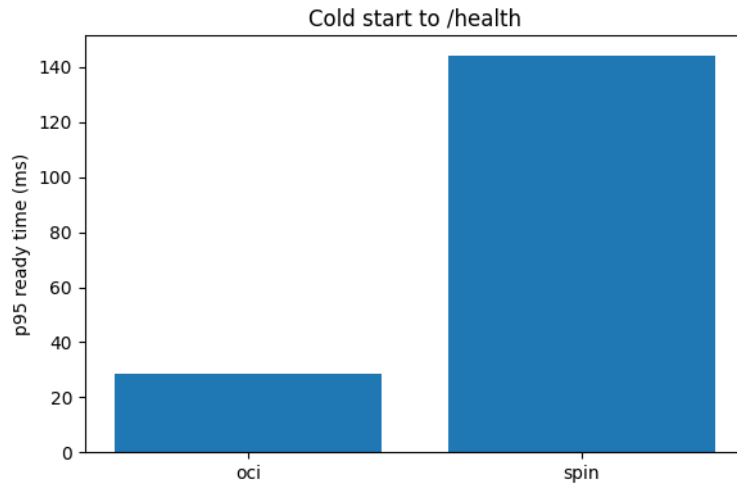
Ważne ograniczenie dotyczy pamięci. OCI jest mierzone przez `docker stats` i odzwierciedla perspektywę kontenera. Spin jest mierzony jako RSS drzewa procesów hosta. Te liczby są użyteczne do porównania obserwowanego narzutu, ale nie są identyczną warstwą rachunkowości pamięci.

5 Wyniki

Wszystkie testy funkcjonalne i obciążeniowe zakończyły się bez błędów walidacji odpowiedzi. Parity check potwierdził identyczne wyniki funkcjonalne dla obu runtime’ów.

Tabela 1: Cold start: 20 powtórzeń na system.

System	Metryka	Mediana [ms]	p95 [ms]	p99 [ms]	Błędy
OCI	/health	20.982	28.420	36.325	0
OCI	/health + /search	307.755	376.983	446.288	0
Spin	/health	141.357	144.220	152.354	0
Spin	/health + /search	205.649	217.504	218.947	0

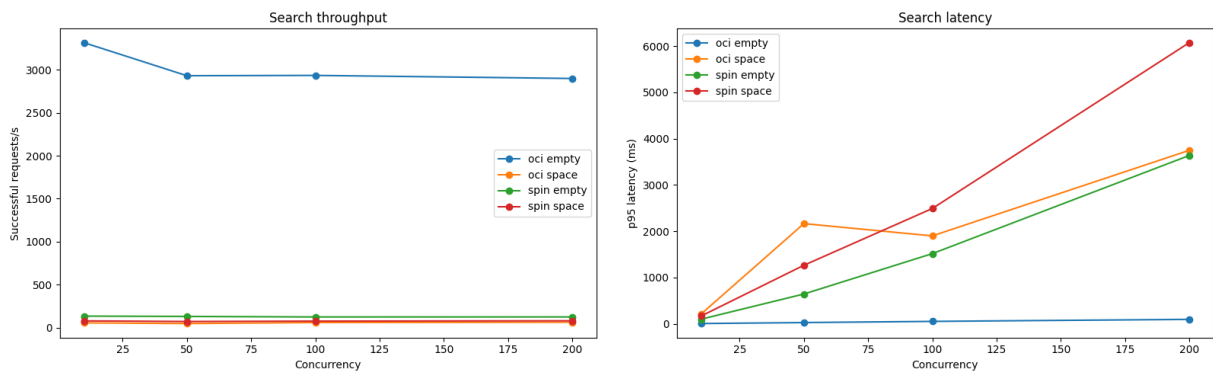


Rysunek 2: p95 cold start do pierwszego poprawnego /health.

W tym środowisku kontener szybciej osiągał gotowość /health, natomiast Spin szybciej wykonywał sekwencję cold start plus pierwsze wyszukiwanie. Wynik ten warto interpretować jako cechę konkretnej implementacji i sposobu pomiaru: /health jest tanim endpointem gotowości, a /search uruchamia realny przepływ pracy aplikacji.

Tabela 2: Load test: request rate i p95 latency.

System	Query	c=10		c=50		c=100		c=200	
		req/s	p95 ms	req/s	p95 ms	req/s	p95 ms	req/s	p95 ms
OCI	empty	3314.9	5.4	2931.1	26.8	2934.6	52.3	2899.0	95.7
Spin	empty	132.8	100.0	129.1	643.5	123.0	1516.1	123.3	3636.0
OCI	space	54.0	210.8	46.2	2164.9	58.0	1899.6	61.5	3747.5
Spin	space	77.2	168.3	70.3	1264.6	74.5	2491.2	78.3	6073.3

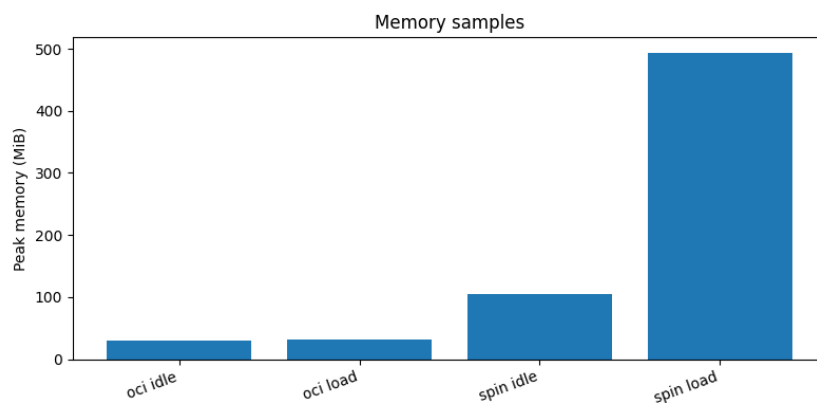


Rysunek 3: Przepustowość i p95 latency dla POST /search.

Dla pustego zapytania natywny serwer OCI był zdecydowanie szybszy. Jest to ścieżka o niskim koszcie aplikacyjnym: zwraca deterministycznie pierwsze dokumenty, bez kosztownego filtrowania tekstu. Dla zapytania **space** różnice są bardziej złożone. Spin osiągał wyższą przepustowość, ale przy bardzo wysokiej współbieżności miał gorszy ogon opóźnień. To sugeruje, że benchmark mierzy nie tylko izolację, lecz także interakcję prostego liniowego skanowania 44 471 rekordów z modelem HTTP i schedulingiem danego runtime'u.

Tabela 3: Pamięć: maksimum zaobserwowanych próbek.

System	Faza	Źródło	Max [MiB]
OCI	idle	<code>docker stats</code>	29.8
OCI	load	<code>docker stats</code>	31.9
Spin	idle	host process RSS	104.8
Spin	load	host process RSS	493.5



Rysunek 4: Maksimum próbek pamięci dla idle i load.

Pamięć jest najmocniejszym argumentem przeciwko prostemu twierdzeniu, że wariant WASM zawsze jest lepszy. W tym projekcie obserwowany proces Spin wraz z wasmtime miał większy RSS niż kontener OCI. Jednocześnie nie jest to pełny model kosztu produkcyjnego, bo Docker raportuje pamięć z perspektywy kontenera, a Spin z perspektywy procesu hosta. W raporcie końcowym oznacza to ostrożny wniosek: Spin daje ciekawy model izolacji capability-oriented, ale wymaga świadomego pomiaru i nie gwarantuje automatycznie niższego narzutu.

6 Demo

Demo prezentacyjne nie uruchamia pełnego benchmarku na żywo. Pełne pomiary są zebrane wcześniej, a w trakcie prezentacji pokazujemy reprodukowalną ścieżkę:

1. uruchomienie Spin na 127.0.0.1:8080 i OCI na 127.0.0.1:8081;
2. sprawdzenie `/health`, `/version` i `/stats`;
3. jedno wyszukiwanie `space`;
4. uruchomienie `benchmarks/compare_results.sh`;
5. pokazanie finalnych wykresów i plików CSV.

Jeżeli środowisko live zawiedzie, plan awaryjny polega na pokazaniu zapisanych artefaktów: `results/raw`, `results/processed` i `results/plots`. Pełny scenariusz znajduje się w `demo/demo-script.md`.

7 Reprodukowalność i ograniczenia

Repozytorium zostało przygotowane tak, aby wynik dało się odtworzyć jednym przebiegiem:

```
benchmarks/run_all.sh
```

Skrypt buduje oba warianty, uruchamia testy poprawności, wykonuje smoke/parity checks, zbiera trzy rodzaje surowych CSV i generuje pliki przetworzone: `results/processed/cold_start_summary.csv`, `results/processed/load_summary.csv` oraz `results/processed/memory_summary.csv`. Wykresy w raporcie pochodzą z `results/plots`. Surowe dane finalnego przebiegu zachowano w `results/raw`; wcześniejsze dane pilota nie są mieszane z wynikami końcowymi.

Najważniejsze ograniczenia interpretacyjne są trzy. Po pierwsze, aplikacja nie używa indeksu odwrotnego: wyszukiwanie `space` jest świadomie prostym skanowaniem tekstowym, więc nie porównuje jakości produkcyjnych wyszukiwarek. Po drugie, pomiar cold start zależy od lokalnego hosta, cache systemowego i tego, że obraz OCI jest zbudowany wcześniej. Po trzecie, pamięć Spin i OCI jest mierzona z różnych perspektyw narzędziowych. Wnioski należy więc traktować jako wynik kontrolowanego eksperymentu lokalnego, a nie uniwersalną charakterystykę wszystkich workloadów WASM i kontenerowych.

8 Wnioski

Najważniejszy rezultat projektu jest metodologiczny: porównanie jest uczciwe, ponieważ oba runtime'y wykonują ten sam kod aplikacyjny i zwracają identyczne wyniki. Spin/WASM dobrze pasuje do małych, izolowanych komponentów HTTP, pluginów, funkcji edge i workloadów serverless, w których istotne są przenośność, sandboxing i prosty model capability-oriented. Kontenery OCI pozostają dojrzsze operacyjnie: mają powszechne narzędzia, stabilną rachunkowość zasobów, znany model wdrożeń i bardzo dobrą wydajność dla prostych ścieżek natywnych.

W naszym benchmarku OCI miało szybszą gotowość `/health`, Spin miał szybszy pierwszy request wyszukiwania po starcie, a wyniki load zależały mocno od zapytania. Puste zapytanie zdecydowanie premiowało OCI, natomiast zapytanie `space` pokazało wyższą przepustowość Spin przy gorszych ogonach opóźnień przy dużej współbieżności. Próba Meilisearch pozostaje wartościowym appendixem: storage-heavy system oparty o LMDB i mmap nie jest dobrym kandydatem do prostego portu WASI bez istotnych zmian architektury.

Literatura

- [1] WebAssembly, *Official website*. <https://webassembly.org/>.

- [2] WASI.dev, *Interfaces*. <https://wasi.dev/interfaces>.
- [3] Bytecode Alliance, *Wasmtime Introduction*. <https://docs.wasmtime.dev/introduction.html>.
- [4] Fermyon, *Develop serverless WebAssembly apps with Spin*. <https://www.fermyon.com/spin>.
- [5] Open Container Initiative, *Open Container Initiative*. <https://opencontainers.org/>.
- [6] Docker, *What is a Container?*. <https://www.docker.com/resources/what-container/>.
- [7] Meilisearch, *Storage*. <https://www.meilisearch.com/docs/resources/internals/storage>.
- [8] Meilisearch, *Backing up Meilisearch data*. https://www.meilisearch.com/docs/resources/self_hosting/data_backup/overview.